

Block-Pool Sizing & KV Transport, Worked Out by Hand

HOW MANY TOKENS FIT ON A GPU — AND HOW FAST A REQUEST’S KV CAN MOVE BETWEEN GPUS

What this document does. It closes Module 04 with the two sizing calculations that turn KV engineering into an RFC you can write without a benchmark cycle. *Part A* sizes the PagedAttention *block pool*: given an HBM budget, the model weights, and an activation reserve, how many KV tokens fit, how many blocks is that, and what maximum concurrency does it support? *Part B* sizes the *KV transport* for disaggregated serving: once prefill and decode run on separate GPU pools, a request’s ~2 GB KV cache must physically move between them before the first decode step — how long does that take over NVLink versus PCIe versus the network? Every figure is produced by a reference script (Appendix A, `m04_t05_engine.py`).

The one idea. Both questions are a single division. Capacity is $\text{pool_bytes} / \text{kv_per_token}$; transfer time is $\text{KV_size} / \text{link_BW}$. Get the units right and the deployment spec writes itself.

CONCEPTS COVERED, IN ORDER

block-pool bytes = HBM – weights – reserve; tokens-in-pool; block count; max concurrency; the H200 worked example (99 GiB → 649k tokens, ~80 concurrent); KV transfer time = KV/BW; and the NVLink-vs-PCIe-vs-RDMA transport table.

INTERNAL LEARNING MATERIAL. NUMBERS ARE REPRODUCIBLE FROM APPENDIX A. v1.0

Contents

1	The two sizing questions	1
2	Part A — Block-pool sizing	2
3	Part B — KV transport for disaggregation	3
4	The argument, at a glance	4
4.1	Equation cheat-sheet	5
	Appendix A — Reference implementation	5

1 The two sizing questions

KV engineering is fundamentally an *allocation* problem. Every mechanism in this module — paging, prefix sharing, eviction, chunking, disaggregation — is a different answer to “how do I allocate scarce KV memory to maximize useful work?” This topic provides the two arithmetic backbones of that decision.

Quantity	Symbol	Value here
H200 HBM	—	141 GiB
Weights (Llama-3-70B, AWQ W4)	—	37 GiB
Activation / workspace reserve	—	5 GiB
KV per token (FP8, GQA-8, 80L)	κ	160 KiB/token
Block size	—	32 tokens
Max context per request	—	8192 tokens
KV per request (for transport)	—	~ 2 GB

A note on units, because this topic mixes them deliberately. HBM, weights, and link bandwidths are quoted the way vendors do: *decimal* (1 GB = 10^9 B, 900 GB/s). The pool-to-tokens division is done in *binary* (1 GiB = 2^{30} , 1 KiB = 1024) because that is how serving engines report block and pool sizes. Each step states which it uses.

2 Part A — Block-pool sizing

Step 1 — Pool bytes

The block pool gets whatever HBM is left after the static weights and a safety reserve for activations and kernel workspace.

Block-pool capacity

$$\text{pool_bytes} = \text{hbm_bytes} - \text{weight_bytes} - \text{activation_reserve}.$$

Plug in the H200 numbers

$$\text{pool} = 141 - 37 - 5 = 99 \text{ GiB}.$$

The activation reserve (3–6 GiB, here 5) is a safety margin: under-reserve and a burst of long prompts OOMs the workspace; over-reserve and you waste KV capacity. It is the one judgment call in an otherwise mechanical calculation.

Step 2 — Tokens in the pool

Divide the pool by the per-token KV rate κ from Topic 1. With FP8 KV, $\kappa = 160$ KiB/token.

Tokens, blocks, concurrency

$$\text{tokens_in_pool} = \frac{\text{pool_bytes}}{\kappa}, \quad \text{num_blocks} = \frac{\text{tokens_in_pool}}{\text{block_size}}, \quad \text{max_conc} = \frac{\text{tokens_in_pool}}{\text{max_tokens/req}}.$$

Capacity in tokens (binary units)

$$\text{tokens_in_pool} = \frac{99 \text{ GiB}}{160 \text{ KiB}} = \frac{99 \times 2^{30}}{160 \times 1024} = 648,806 \approx \mathbf{649k} \text{ tokens.} \quad \checkmark$$

Step 3 — Blocks and concurrency

Blocks and max concurrency

$$\begin{aligned}\text{num_blocks} &= \frac{648,806}{32} = 20,275 \text{ blocks,} \\ \text{max_concurrency} &= \frac{648,806}{8192} = 79.2 \approx \mathbf{80} \text{ max-context requests.}\end{aligned}$$

At the Nexus working length of 6200 tokens (not the 8k ceiling), the same pool holds $648,806/6200 \approx 105$ concurrent requests.

One H200 serves ~80 concurrent requests

A single H200, after AWQ weights and FP8 KV, holds a 99 GiB pool = ~649k KV tokens = ~80 concurrent max-context requests — which matches the Nexus v4 observed ~80 concurrent 6.2k-requests-per-H200. Every lever in this module feeds this one number: AWQ shrank the weights (more pool), FP8 halved κ (more tokens), paging eliminated fragmentation (more usable tokens), prefix sharing freed duplicate prefixes (more *effective* tokens). The block-pool sizing is where they all cash out.

Watch out

The two big levers in the token count are κ and the weights. Had we kept FP16 KV ($\kappa = 320$ KiB), the pool would hold only ~324k tokens and ~40 concurrent requests — half. Had we skipped AWQ (≈ 70 GiB of FP16 weights), the pool would shrink to ~66 GiB. Quantizing *both* weights and KV is what makes 80-way concurrency fit on one GPU.

3 Part B — KV transport for disaggregation

Disaggregated serving (Topic on disaggregation) runs prefill and decode on separate, independently-sized GPU pools. The catch: after the prefill pool finishes a request, the *entire* KV cache — ~2 GB for Nexus — must move to the decode pool before the first decode step can run. That transfer happens *on every single request* and is pure added latency before the first token.

KV transfer time

$$\text{transfer_time} = \frac{\text{KV_size}}{\text{link_bandwidth}}.$$

Step 4 — The transport table

Move 2 GB over each candidate link (bandwidths in decimal GB/s):

Two GB over each link

$$\text{NVLink (same node, 900 GB/s)} : \frac{2 \text{ GB}}{900 \text{ GB/s}} = 2.2 \text{ ms},$$

$$\text{PCIe 5.0} \times 16 \text{ (64 GB/s)} : \frac{2 \text{ GB}}{64 \text{ GB/s}} = 31.2 \text{ ms},$$

$$\text{RDMA IB 400 Gb (50 GB/s)} : \frac{2 \text{ GB}}{50 \text{ GB/s}} = 40.0 \text{ ms},$$

$$\text{TCP/IP (10 GB/s)} : \frac{2 \text{ GB}}{10 \text{ GB/s}} = 200 \text{ ms}.$$

Transport	Bandwidth	Time (2 GB)	Use case
NVLink (same node)	900 GB/s	2.2 ms	pools on the same physical host ✓
PCIe 5.0 × 16	64 GB/s	31.2 ms	pools across the PCIe fabric
RDMA IB / RoCE	50 GB/s	40 ms	different nodes, tight datacenter
TCP/IP	10 GB/s	200 ms	don't — never — not production

NVLink hides the transfer; the network spends your budget

NVLink moves a request's 2 GB KV in **2.2 ms** — negligible against a 72 ms TTFT target. PCIe 5 takes **~32 ms** (~14× slower), a real but survivable bite. RDMA at 40 ms eats most of a tight budget, and TCP/IP at 200 ms would burn it entirely. **The rule: co-locate prefill and decode pools on the same NVLink domain.** If they must span nodes, RDMA is the floor of acceptability and TCP/IP is disqualifying.

Watch out

This transfer cost is exactly why disaggregation is not always worth it. For Nexus at 10k req/s with long 6200-token prompts, the scheduling win dwarfs a 2–32 ms transfer. For a chat service at 500 req/s with 50-token prompts, the per-request KV transfer overhead *exceeds* the scheduling win, and a well-tuned cohabitated system with chunked prefill (Topic 4) is strictly better. Rule of thumb: disaggregate only when the region has > 4 GPUs on the model and the prefill:decode compute ratio is more lopsided than 3:1.

4 The argument, at a glance

#	Step	Result
A1	Pool bytes	$141 - 37 - 5 = 99 \text{ GiB}$
A2	Tokens	$99 \text{ GiB} / 160 \text{ KiB} = 649\text{k}$
A3	Blocks	$649\text{k} / 32 = 20,275$
A4	Concurrency	$649\text{k} / 8192 \approx 80$ ✓
B1	Transfer	KV/BW
B2	NVLink vs PCIe	2.2 ms vs 31 ms (~14×)

4.1 Equation cheat-sheet

Pool bytes	$\text{pool} = \text{hbm} - \text{weights} - \text{reserve}$
Tokens in pool	$\text{tokens} = \text{pool_bytes} / \kappa$
Block count	$\text{num_blocks} = \text{tokens} / \text{block_size}$
Max concurrency	$\text{max_conc} = \text{tokens} / \text{max_tokens_per_req}$
KV transfer time	$\text{transfer_time} = \text{KV_size} / \text{link_BW}$

Appendix A — Reference implementation

Every number above is produced by `m04_t05_engine.py` (provided alongside this PDF). Run it to reproduce all figures; change the HBM budget, dtype, or link bandwidths to size any GPU / transport combination.

```

1  """
2  Reference implementation for: Module 04 - Topic 5
3  "Block-Pool Sizing & KV Transport - how many tokens fit, and how fast KV moves."
4
5  Every number printed here is transcribed (rounded) into the worked-by-hand PDF,
6  so the arithmetic in the document is exact and self-consistent.
7
8  Two linked questions:
9      (A) Block-pool sizing: given an HBM budget, weights, and an activation reserve,
10         how many KV tokens fit, how many blocks, and what max concurrency?
11      (B) KV transport for disaggregated prefill/decode: how long does it take to move
12         one request's KV across NVLink vs PCIe5 vs RDMA?
13
14  Unit convention is mixed and stated at each step: HBM/weights in DECIMAL GB (1e9),
15  the pool->tokens division in BINARY (GiB / KiB) the way engines report it, and the
16  transport bandwidths in DECIMAL GB/s as vendors quote links.
17  """
18
19  # =====
20  # PART A - BLOCK POOL SIZING (H200, Llama-3-70B AWQ, FP8 KV)
21  # =====
22  hbm_GiB      = 141      # H200 HBM
23  weights_GiB  = 37       # Llama-3-70B after AWQ W4
24  act_reserve_GiB = 5      # activation / workspace safety reserve
25  block_size   = 32       # tokens per block
26  kv_per_token_B = 160 * 1024 # FP8 KV, Llama-3-70B GQA-8 => 160 KiB/token (Topic 1)
27  max_tokens_req = 8192    # max context per request
28
29  # ---- 1. Pool bytes = HBM - weights - activation reserve ----
30  pool_GiB = hbm_GiB - weights_GiB - act_reserve_GiB
31  print("== A1. Block-pool bytes ==")
32  print(f"pool = {hbm_GiB} - {weights_GiB} - {act_reserve_GiB} = {pool_GiB} GiB")
33
34  # ---- 2. tokens_in_pool = pool_bytes / kv_bytes_per_token ----
35  pool_bytes = pool_GiB * (2**30)      # binary GiB -> bytes
36  tokens_pool = pool_bytes / kv_per_token_B # both binary -> clean
37  print("\n== A2. Tokens that fit in the pool ==")
38  print(f"tokens_in_pool = {pool_GiB} GiB / 160 KiB-per-token = {tokens_pool:,.0f} tokens "
39        f"({tokens_pool/1e3:,.0f}k)")
40
41  # ---- 3. num_blocks = tokens_in_pool / block_size ----
42  num_blocks = tokens_pool / block_size
43  print("\n== A3. Block count ==")
44  print(f"num_blocks = {tokens_pool:,.0f} / {block_size} = {num_blocks:,.0f} blocks")
45

```

```

46 # ---- 4. max_concurrency = tokens_in_pool / max_tokens_per_request ---
47 max_conc = tokens_pool / max_tokens_req
48 print("\n== A4. Max concurrency ==")
49 print(f"max_concurrency = {tokens_pool:,.0f} / {max_tokens_req} = {max_conc:.1f} "
50       f"concurrent max-context requests")
51 # at the Nexus 6.2k working length
52 conc_6200 = tokens_pool / 6200
53 print(f" (at the 6.2k Nexus working length: {conc_6200:.0f} concurrent requests)")
54
55 # =====
56 # PART B - KV TRANSPORT for DISAGGREGATED prefill/decode
57 # =====
58 # After prefill, one request's KV (~2 GB) must move from the prefill pool to
59 # the decode pool before the first decode step. transfer_time = KV_size / BW.
60 kv_size_B = 2e9          # ~2 GB of KV for one Nexus request (decimal GB)
61 print("\n== B1. KV transfer time = KV_size / link_bandwidth ==")
62 print(f"KV to move per request = {kv_size_B/1e9:.1f} GB")
63
64 links = [
65     ("NVLink (same node)",      900e9),    # 900 GB/s
66     ("PCIe 5.0 x16",           64e9),      # 64 GB/s
67     ("RDMA InfiniBand (400 Gb)", 50e9),     # 50 GB/s
68     ("TCP/IP",                 10e9),      # 10 GB/s (don't)
69 ]
70 for name, bw in links:
71     t_ms = kv_size_B / bw * 1e3
72     print(f" {name:26s}: {kv_size_B/1e9:.0f} GB / {bw/1e9:.0f} GB/s = {t_ms:6.1f} ms")
73
74 # ---- The headline contrast: NVLink vs PCIe5 -----
75 nvlink_ms = kv_size_B / 900e9 * 1e3
76 pcie_ms   = kv_size_B / 64e9  * 1e3
77 print("\n== B2. The headline contrast ==")
78 print(f"NVLink: ~{nvlink_ms:.1f} ms   vs   PCIe5: ~{pcie_ms:.0f} ms  "
79       f"({pcie_ms/nvlink_ms:.0f}x slower)")
80 print("On every single request, the KV transfer is pure added latency before")
81 print("the first decode token. NVLink hides it; PCIe5 spends ~32 ms of budget;")
82 print("TCP/IP (200 ms) would burn the entire latency budget. Co-locate the pools.")

```

— END OF DOCUMENT —